ZevaPay Frontend: Phased Build Prompts

Nov 3, 2026 · @Abdullah

How to use + ground rules

Ekta phase ekbar e paste korun. Prottek phase er age nicher ground rules block ta paste korben. Phase shesh hole lint, typecheck ar build pass kora lagbe, tarpor porer phase.

Ground rules (paste before every phase)

- Work only inside `frontend/`. Do not touch `backend/`, `db/` or root config. If a change outside `frontend/` is truly needed, stop and ask first.
- Inspect before coding. Read `frontend/AGENTS.md` (Next.js 16 has breaking changes, so check `node_modules/next/dist/docs/` first), `package.json` and the existing components. Reuse them. Add no new dependency without saying why.
- Keep the current routes working (`/dashboard`, `/dashboard/accounts`, `/dashboard/devices`, `/dashboard/transactions`, `/admin/...`). Pages the backend already supports keep calling the real API.
- Every other page reads from a mock layer in `src/mock/`, exposing the same function shapes as `src/lib/api-client.ts`, so it can be swapped for the real API later.
- Show a visible "Demo data" badge on every mock-backed page.
- An SMS match is not proof of payment. Never label a payment as verified because the parser was confident.
- Mock data is fixed, never random per render. Money in BDT (৳), timezone Asia/Dhaka.
- No real payment processing, no real MFS connection, no PIN or OTP fields.
- End each phase with a short list of files added and changed, then stop.

Real API vs mock

The current backend covers accounts, devices, transactions, review and API keys. Everything else in the spec has no database model yet, so it is mock-only for now.

Area	Backend today	Build as
Overview, transactions, review queue	Real	Real API
Payment accounts	Real	Real API
Devices and pairing	Real	Real API
API keys	Real (inside Settings)	Real API, moved to the API page
Admin: merchants, users, devices, transactions	Real	Real API
Team and roles	Partly (users with fixed roles)	Real list, mock invite and custom roles
Reports	Partly (derivable from transactions)	Mock first
Customers, payment links, invoices, brands, checkout builder, webhooks, sandbox	No model	Mock only
Notifications, billing, plans, providers, parsers, feature flags, system health	No model	Mock only
Customer checkout and payment pages	No payment or order model	Mock only

The spec has 8 payment statuses and the backend has 4. Use this mapping everywhere:

Spec status	Backend status	Label to show
CREATED	none	Created (mock only)
PENDING	PENDING	Awaiting verification
REVIEW	REVIEW	Needs review
CONFIRMED	SUCCESS	Confirmed from SMS, not verified with the provider

Spec status	Backend status	Label to show
FAILED	FAILED	Verification failed
EXPIRED, CANCELLED, REFUNDED	none	Mock only

Phase 1: Foundation

Shared look and plumbing only. No new feature pages yet.

```
Phase 1. Inspect the repo and report the frontend stack in five lines.

Then build the foundation:
1. Design tokens for light and dark themes (navy primary, emerald accent,
neutral background), using the existing Tailwind setup.
2. Extend src/components/ui/ only where something is missing: tabs, drawer,
tooltip, empty state, skeleton.
3. Three layouts: public, merchant dashboard (grouped sidebar: Overview,
Collection, Automation, Business, General) and admin (its own sidebar).
4. src/mock/ with a small in-memory helper for fixed mock records, a "Demo
data" badge, and a status badge that follows the status mapping.
5. Theme toggle and collapsible sidebar.

Done when: existing pages still work, the theme toggle works, the sidebar
shows every group (unbuilt links may point to a "Coming in a later phase"
page), and lint, typecheck and build pass.
```

Phase 2: Public website and authentication

Login, register and forgot-password already exist and use the real API, so they stay. This phase adds the missing public pages and the UI-only auth screens.

```
Phase 2. Build the public website and the missing auth screens.

Public: improve the landing page (hero, supported methods, how it works,
security, pricing preview, FAQ, contact) and add /features, /pricing and
/status. Pricing is demo content, labelled as configurable. Do not claim
provider partnerships, regulatory approval or guaranteed verification.

Auth: keep the real /login, /register and /forgot-password. Add UI-only
/reset-password, /verify-email and /two-factor, each labelled as not
connected to the backend. Add a password strength indicator and a show/hide
toggle to the existing forms.

Demo role switch (Merchant or Admin): only when NEXT_PUBLIC_DEMO_MODE=true,
never in a production build, and it must not set real session cookies.

Done when: every public page is reachable from the header or footer, forms
validate, both themes look right on mobile and desktop, and lint, typecheck
and build pass.
```

Phase 3: Merchant core

The pages a merchant uses every day. Most of these already have real data behind them, so improve the UI and keep the real API.

```
Phase 3. Build the merchant core inside the existing /dashboard.

1. Overview: stat cards, date filter (Today, 7, 30, 90 days, custom), revenue and volume charts, payment method split, recent transactions, pending review alert, device status.
2. Payments: list and detail with a timeline, internal notes, and approve, reject or refund as clearly labelled demo actions. Until the backend has a payment model, show transactions here.
3. Transactions: search, status, provider, amount and date filters, sorting, pagination, CSV export and a detail drawer.
4. Payment accounts: keep the real list; add label, type, priority and enable toggle. Show limits and health as demo fields. Always mask account numbers.
5. Devices: keep the real list; add a pairing code modal with a QR preview. State clearly that SMS forwarding is simulated here.
6. Customers (mock): directory, detail page, notes, tags, CSV import and export UI.

Done when: real pages still load real data, mock pages carry the Demo badge, search, filters, sorting and pagination work, and lint, typecheck and build pass.
```

Phase 4: Collection and developer tools

All mock-only except API keys, which already exist in the backend.

```
Phase 4. Build the collection and developer pages inside /dashboard. Use the
mock layer, except API keys.

1. Payment links: list, create and edit form, copy link, QR preview, enable
or disable, expiry and usage limits.
2. Invoices: list, create form with line items, tax and discount totals,
status, a PDF preview UI and a demo reminder action.
3. Brands and websites: list, create and edit, logo and color, assigned
accounts. Support several brands per merchant.
4. Checkout builder: form-based settings (logo, colors, theme, method order,
instructions, redirects) with a live preview and Save draft.
5. API and developers: keys on the real API (secret shown once, then masked),
a usage chart and demo request logs.
6. Webhooks: endpoint list, event selection, masked secret, delivery logs
with request and response preview, demo retry.
7. Sandbox: simulate success, failed and pending payments and a test webhook.
Keep sandbox data visually separate from live data.

Done when: create, edit and delete work locally (delete asks for
confirmation), forms validate, every mock page carries the Demo badge, and
lint, typecheck and build pass.
```

Phase 5: Business and settings

Reports, notifications, team, billing, settings and security.

```
Phase 5. Build the business and settings pages inside /dashboard.

1. Reports: revenue, transactions, payment methods, accounts, refunds, fees, success and failure, reconciliation. Summary cards, charts, date range, CSV export and an Export Excel button that shows a demo notice.
2. Notifications: channel cards (Email, Dashboard, SMS, Telegram, Discord, Slack, Webhook) and an event preference grid. Nothing is actually sent, and the page says so.
3. Team: member list from the real users, plus mock invite, a role permission matrix (Owner, Admin, Manager, Support, Viewer) and custom roles.
4. Billing: current plan, monthly or yearly toggle, plan comparison, usage limits, billing history, cancel UI. All demo.
5. Settings: business profile, regional preferences (BDT, Asia/Dhaka, English and Bangla labels), checkout preferences.
6. Security: password change, two-factor UI, active sessions and login history from the real data where it exists, account deactivation with a confirmation dialog.

Done when: every page has real structure and mock data (no empty placeholders), CSV exports download a usable file, and lint, typecheck and build pass.
```

Phase 6: Super admin dashboard

A separate layout under `/admin`. Use the real API wherever an endpoint already exists, and mock the rest.

```
Phase 6. Build the super admin dashboard under /admin with its own sidebar
(Overview, Operations, Developers, Management, System). Keep the existing
/admin pages working. Use the real API where an endpoint exists (merchants,
transactions, devices, review queue); mock the rest.

1. Overview: merchant counts by status, transaction volume, success rate,
active and offline devices, webhook failures, charts and recent activity.
2. Merchants: list, profile, approve, reject, suspend, reactivate, plan
assignment, limits and internal notes.
3. Payment reviews: queue with the submitted TrxID, amount, provider,
merchant and customer, review notes, approve or reject, and review history.
4. Providers and parsers (mock): enable or disable, limits, fees, parser
version, integration health.
5. Devices: all devices, last sync, parser errors, duplicate events, revoke.
6. API logs, webhook monitoring, plans, notifications, staff, audit logs,
security, system health and feature flags (mock).

Done when: an admin cannot reach merchant pages and a merchant cannot reach
/admin, every sidebar link opens a real page, and lint, typecheck and build
pass.
```

Phase 7: Checkout, docs portal and final QA

The customer-facing checkout, the developer docs, and the final check across the whole app.

Phase 7. Build the demo checkout, the docs portal and run the final QA.

1. Checkout (mock, mobile-first) at /checkout/[paymentId] with /processing, /success, /failed and /expired. Show the merchant logo and name, order summary, method selection (bKash, Nagad, Rocket, Upay), numbered payment instructions, the merchant account number, phone and TrxID inputs with validation, a retry, and a return-to-merchant button. Label it as a demo.
2. Docs portal at /docs: sidebar, search UI, Introduction, Quick Start, Authentication, Create Payment, Webhooks, Error Codes, and code tabs for Node.js, PHP, Python and cURL with a copy button. Mark every example as illustrative until the backend API exists.
3. QA: open every route and check navigation, forms, modals, filters, sorting, pagination, CSV export, theme switch, sidebar collapse and keyboard focus on mobile and desktop.
4. Run lint, typecheck and the production build, fix what your changes broke, and confirm backend/, db/ and root config are unchanged (git diff).

Done when: the checklist above passes and you list the remaining limitations.